ЦЕНТРАЛЬНЫЙ УНИВЕРСИТЕТ

Стек протоколов. Уровень представления

Разработка
Сетевые технологии

02



Сетевые технологии · лекция 2

Представление данных, TLS и Web-протоколы

Что находится внутри сообщения
и почему мы доверяем другой стороне

Наше место в модели OSI

Спускаемся от прикладного уровня к представлению данных: второй уровень сверху, L6.

Уровень	Название	Где мы в курсе
L7	Прикладной	Лекция 1: HTTP и другие прикладные диалоги
L6	**Представления**	**Сегодня: формат данных и защита обмена**
L5	Сеансовый	
L4	Транспортный	
L3	Сетевой	
L2	Канальный	
L1	Физический	

> OSI разделяет функции. Реальный протокол может выполнять задачи нескольких уровней.

Данные доставлены. Что находится внутри?

В первой лекции мы подробно разобрали HTTP, затем познакомились с другими прикладными протоколами.

Протокол	Что уже умеем объяснить	Следующий вопрос
HTTP	Запрос ресурса и ответ сервера	Как прочитать и проверить тело ответа?
SMTP	Передача и приём письма сервером	Где текст, HTML и вложения?
IMAP / POP3	Доступ к письмам в ящике	Как отобразить полученное письмо?

> В разных протоколах возникает общая задача: как интерпретировать содержимое сообщения?

Представление данных: задачи и механизмы

Для работы с содержимым нужны разные форматы, стандарты и протоколы.

Задача	Примеры механизмов	Где встречаем
Представить текст	UTF-8: кодирование символов	Почта, HTTP, сообщения сервисов
Описать содержимое и части	MIME: типы и структура сообщения	Почта, составные HTTP-тела
Передать структуру данных	JSON, Protobuf, ASN.1	API, RPC, управление сетью
Защитить обмен	TLS: защищённое соединение	Web, почта и другие сервисы

> Одна задача может встречаться в разных прикладных протоколах.

Что разберём на этом занятии

От общей карты перейдём к чтению сообщений и проверке их обработки.

Вопрос	Результат
Как интерпретировать байты?	Определить тип, кодировку символов и сжатие
Как устроено составное сообщение?	Найти части сообщения и их метаданные
Как описывают структуру данных?	Различить схему и байтовое кодирование
Как проверяют другую сторону?	Объяснить TLS и проверки сертификата
Что означает ошибка клиента?	Назвать этап и следующую проверку

Почему начнём с электронной почты

В одном письме наглядно сочетаются текст, HTML, картинки и вложения. На нём удобно разобрать общие задачи представления.

Содержимое письма	Что должна знать программа
Текст на русском языке	Какие байты обозначают буквы?
HTML-версия	Это другой документ или версия того же текста?
Картинка в тексте	Какая часть письма содержит её данные?
Файл отчёта	Где границы вложения и какой у него тип?

> Разберём эти задачи через MIME, затем вернёмся к их применению в HTTP.

MIME расширяет формат письма

MIME (Multipurpose Internet Mail Extensions) описывает типы содержимого, кодирование и составные сообщения.

Историческое ограничение почты	Механизм MIME
Ориентация на строки US-ASCII	Объявление charset и кодирование тела
Письмо с одним неструктурированным телом	Несколько частей с собственными заголовками
Получателю неизвестен формат данных	Media type: тип и подтип содержимого

> SMTP переносит письмо. MIME помогает получателю разобрать его содержимое.

Письмо как последовательность строк

```
From: sender@example.org
To: student@example.net
Subject: Course example
MIME-Version: 1.0
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: base64

0J/RgNC40LLQtdGC
```

Заголовки	Пустая строка	Тело
Описывают сообщение и содержимое	Отделяет заголовки от тела	Base64-декодирование даёт UTF-8-байты слова «Привет»

Multipart: каждая часть имеет свои заголовки

```
Content-Type: multipart/mixed; boundary="part"

--part
Content-Type: text/plain; charset=US-ASCII

See attachment.
--part
Content-Type: text/plain; charset=US-ASCII
Content-Disposition: attachment; filename="report.txt"

OK
--part--
```

Объявление	Разделитель части	Конец multipart
boundary="part"	--part	--part--

Подтипы multipart задают отношения частей

Подтип	Как связаны части	Пример
`mixed`	Разные части одного сообщения	Текст и вложение
`alternative`	Альтернативы одного содержимого	Один текст в plain и HTML
`related`	Взаимосвязанные ресурсы	HTML и встроенные картинки
`form-data`	Значения полей формы	Имя пользователя и файл

> Разделители похожи, но получатель по-разному использует части.

Тип содержимого и способ отображения

Поле	Что сообщает	Пример
`Content-Type`	Формат содержимого	image/png
`charset`	Кодировка текстовых байтов	UTF-8
`Content-Disposition`	Подсказка по отображению	attachment; filename="plot.png"
`Content-ID`	Идентификатор части	<plot@example.org>

> HTML может сослаться на часть как `src="cid:plot@example.org"`.

Кодирование для почтового транспорта

Content-Transfer-Encoding	Что происходит с байтами
`7bit / 8bit / binary`	Преобразования нет. Значение описывает допустимый вид содержимого
`quoted-printable`	Часть байтов записывается как =HH, например =D0=9F
`base64`	Три байта превращаются в четыре символа из ограниченного алфавита

Base64 увеличивает объём примерно на треть, без учёта переносов строк и заполнения.

> Для восстановления данных нужен алгоритм декодирования, секретного ключа нет.

Разбор: письмо выглядит как бессмысленная строка

```
Content-Type: text/plain; charset=UTF-8
Content-Transfer-Encoding: base64

0J/RgNC40LLQtdGC
```

Шаг 1	Шаг 2	Шаг 3
Прочитать метаданные	Восстановить байты из Base64	Декодировать UTF-8

Что изменится, если получатель пропустит второй шаг? А если выберет другую кодировку?

Символы и байты

Кодировка определяет, как записать текст в байтах. Возьмём одно слово: «Привет».

Кодировка	Байты слова «Привет» в hex	Размер
UTF-8	`D0 9F D1 80 D0 B8 D0 B2 D0 B5 D1 82`	12 байтов
UTF-16 LE	`1F 04 40 04 38 04 32 04 35 04 42 04`	12 байтов
Windows-1251	`CF F0 E8 E2 E5 F2`	6 байтов
ASCII	Кириллицу представить нельзя	—

Для UTF-16 выбран порядок «младший байт первым» (LE), без служебной метки.

> Текст один, байты разные. Получателю нужно знать кодировку отправителя.

Те же механизмы за пределами почты

Механизм	Почта	HTTP
Media type	Тип части письма	Тип представления ресурса
Multipart	Текст и вложения	Поля и файлы формы
Content-Disposition	Вложение с именем	Подсказка браузеру о скачивании

Теперь используем знакомый HTTP как второй пример представления данных.

> Сходные поля не означают, что правила всех протоколов одинаковы.

HTTP-ответ описывает своё содержимое

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=UTF-8
Content-Length: 12

Привет
```

Статус	Content-Type / charset	Content-Length
Результат HTTP-запроса	Как прочитать тело	Длина тела в байтах: 12

> Эта длина относится к байтам тела, без заголовков и пустой строки.

Три похожих названия, разные задачи

Поле	Контекст	Пример и смысл
`Content-Transfer-Encoding`	MIME в почте	base64: представление для ограничений транспорта
`Content-Encoding`	HTTP-представление	gzip: преобразование содержимого
`Transfer-Encoding`	HTTP/1.1-сообщение	chunked: передача тела порциями

> Charset описывает текстовые байты и решает ещё одну, отдельную задачу.

Согласование представления

Клиент сообщает предпочтения. Ответ сообщает фактический выбор сервера.

Фрагмент запроса

```
Accept: application/json
Accept-Encoding: gzip
```

Фрагмент ответа

```
Content-Type: application/json
Content-Encoding: gzip
```

> Сначала распаковываем gzip. Затем читаем JSON в UTF-8 и разбираем его структуру.

HTTP-форма с полем и файлом

Запрос клиента к серверу. Фрагмент заголовков и тела.

```
Content-Type: multipart/form-data; boundary="form"

--form
Content-Disposition: form-data; name="student"

Alex
--form
Content-Disposition: form-data; name="file"; filename="a.txt"
Content-Type: text/plain

OK
--form--
```

name	filename	boundary
Имя поля формы	Имя приложенного файла	Границы частей тела

Где заканчивается тело HTTP/1.1?

Получение строки статуса ещё не означает получение полного сообщения.

Ситуация	Как определить границу
Задан Content-Length	Прочитать указанное число байтов
Transfer-Encoding: chunked	Прочитать чанки, последний чанк и завершение trailers
Ответ, ограниченный закрытием	Читать до закрытия соединения
Ответ без тела	Учитывать метод и статус, например HEAD или 204

> Один вызов чтения может вернуть только часть заголовка или тела.

Chunked: длина каждой порции известна

```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked

4
Wiki
5
pedia
0
```

4 и 5	0 и пустая строка	Восстановленное тело
Размеры чанков в hex	Последний чанк, trailers отсутствуют	Wikipedia, 9 байтов

Порядок обработки полученного тела

Этап	Что делаем	Что может пойти не так
Границы сообщения	Собираем тело, снимаем chunked	Обрыв, неверная длина
Content-Encoding	Распаковываем, если задан gzip	Повреждённые сжатые данные
Текстовое представление	Декодируем charset, разбираем JSON/XML	Неверная кодировка или синтаксис
Бинарное представление	Передаём байты декодеру PDF, изображения, DER	Неверная структура формата

> Текст и бинарные форматы используют разные ветви обработки.

Один статус 200, разные результаты

Наблюдение	Совместимая причина	Что сравнить
Вместо JSON пришёл HTML	Ответ прокси, вход в систему или другой ресурс	Content-Type и начало тела
JSON обрывается посередине	Неполное HTTP-тело или уже испорченный JSON	Границу сообщения и байты
Вместо букв видны странные символы	Ошибочная текстовая интерпретация	Charset и исходные байты
gzip не распаковывается	Тело не соответствует объявленному кодированию	Метаданные и кодированное тело

Какая проверка отличит сбой доставки тела от полного, но некорректного JSON?

Структура данных и сериализация

Сериализация задаёт представление структуры данных для обмена между системами.

Формат / семейство	Как описываем структуру	Где встречаем
JSON	Объекты, массивы, значения	API, очереди сообщений
XML	Элементы и атрибуты	Документы, SOAP
Protocol Buffers	Сообщения и поля в .proto	RPC, обмен сервисов
ASN.1	Типы и правила кодирования	Сертификаты, сети, связь

> Корректный синтаксис ещё не гарантирует ожидаемые поля и значения.

ASN.1 описывает типы и ограничения

```
Person ::= SEQUENCE {
  name  UTF8String,
  age   INTEGER (0..150),
  email UTF8String OPTIONAL
}
```

SEQUENCE	INTEGER (0..150)	OPTIONAL
Составная структура	Тип и допустимый диапазон	Поле может отсутствовать

> Для передачи нужны и схема, и выбранные правила кодирования.

TLV: тип, длина, значение

В BER/DER значение часто удобно читать как Tag, Length, Value.

```
02 01 2A
```

Tag: 02	Length: 01	Value: 2A
Универсальный тип INTEGER	Один байт значения	Число 42

> Это простой пример. Тег и длина могут занимать несколько байтов.

Одна схема, разные правила кодирования

Правила	Основная идея	Когда это полезно
Basic Encoding Rules	Гибкий набор допустимых кодирований	Совместимый обмен структурами
Distinguished Encoding Rules	Канонические ограничения BER	Однозначные байты для подписываемых структур
Packed Encoding Rules	Компактное кодирование с учётом схемы	Экономия в протоколах связи
XML Encoding Rules JSON Encoding Rules	Кодирование через XML / JSON	Текстовое представление ASN.1-данных

> OID обозначает объект или алгоритм, например алгоритм подписи сертификата.

ASN.1 встречается далеко за пределами Web

Область	Примеры	Что описывают
Доверие и идентичность	X.509, CSR	Сертификаты и запросы на выпуск
Управление сетью	SNMP	Управляющие сообщения и значения объектов
Каталоги	LDAP	Запросы и ответы службы каталогов
Сигнализация связи	MAP / CAP	Данные услуг и сигнализации
Радиодоступ	RRC	Управление радиосоединением

> Знать место технологии полезно даже без умения настраивать эти системы.

Содержимое понятно. Можно ли доверять отправителю?

Сообщение идёт через чужие сети. TLS (Transport Layer Security) защищает обмен между участниками соединения.

Риск	Нужное свойство
Кто-то читает передаваемые данные	Конфиденциальность
Кто-то изменяет данные по дороге	Целостность
Клиент подключился к подменённому сервису	Аутентификация другой стороны

> TLS защищает соединение. PKI помогает связать ключ с ожидаемым именем.

Механизмы защиты могут работать на разных уровнях OSI. В этой лекции рассматриваем их только в контексте L6 — уровня представления.

Как TLS защищает наш обмен

Открываем `https://api.course.test`. Перед HTTP-запросом клиент устанавливает защищённое соединение с сервером.

Вопрос	Какие механизмы работают вместе
Кому мы доверяем?	Клиент проверяет сертификат и имя сервера. Сервер доказывает владение приватным ключом
Как получить общий секрет?	Клиент и сервер выполняют обмен с временными ключами, не передавая сам секрет
Как защитить сообщения?	Из секрета получают ключи для шифрования данных и проверки их целостности

> Теперь разберём ключи, сертификаты и доверие, затем соберём их в TLS handshake — установление защищённого соединения.

Ключи, согласование секрета и подпись

Механизм	Что делает	Роль в обычном TLS 1.3
Симметричная защита	Стороны используют общие секретные ключи	Шифрование данных и проверка целостности
Согласование ключей	Стороны получают общий секрет по открытому каналу	Эфемерный обмен (EC)DHE
Цифровая подпись	Приватный ключ подписывает, публичный проверяет	Подтверждение владения ключом

> Публичный ключ можно передавать. Приватный ключ должен оставаться секретным.

DHE — Diffie–Hellman Ephemeral.
ECDHE — Elliptic Curve Diffie–Hellman Ephemeral (вариант на эллиптических кривых).
Ephemeral («эфемерный»): временные ключи согласования для конкретного соединения.

PKI связывает имя с публичным ключом

PKI (Public Key Infrastructure) включает сертификаты, центры сертификации и правила доверия.

Участник	Ответственность
Владелец сервиса	Хранит приватный ключ и запрашивает сертификат
CA, центр сертификации	Проверяет основания для выпуска и подписывает сертификат
Клиент	Решает, каким корням доверять, и проверяет предъявленный сертификат

> Наличие сертификата само по себе не создаёт доверие клиента.

Что находится в сертификате X.509

Сертификат публичен. Он связывает идентичность с публичным ключом и подписью издателя.

Поле	Учебный пример
Имена в Subject Alternative Name (SAN)	DNS: api.course.test
Период действия	notBefore … notAfter
Публичный ключ	Ключ для проверки владения приватным ключом
Издатель и подпись	Course Intermediate CA
Назначение и ограничения	TLS server authentication, CA:FALSE

> DER хранит структуру в байтах. PEM оборачивает DER в Base64 с текстовыми метками.

Цепочка до доверенного корня

Сертификат	Кто подписывает	Откуда его берёт клиент
Leaf: api.course.test	Промежуточный CA	Обычно присылает сервер
Intermediate CA	Корневой CA или другой intermediate	Обычно присылает сервер
Root CA	Обычно самоподписанный	Заранее доверен в ОС или приложении

Клиент проверяет подписи и ограничения по пути к своему доверенному корню.

> Присланный сервером корень автоматически доверенным не становится.

Проверки сертификата на стороне клиента

Проверка	Вопрос
Цепочка и доверенный корень	Можем ли подтвердить подписи до своего trust anchor?
Имя сервиса	Соответствует ли сертификат имени из URL?
Время	Попадаем ли в период действия?
Назначение и ограничения	Разрешено ли использовать сертификат в этой роли?
Отзыв	Что требует политика клиента и какие данные доступны?

> Отключение проверки сертификата убирает существенную часть защиты TLS.

Сертификат имеет жизненный цикл

Этап	Что происходит	Возможная проблема
Ключ и CSR	Создание ключа и запроса на сертификат	Утечка private key
Проверка и выпуск	CA проверяет запрос и подписывает сертификат	Неверные имена или назначение
Развёртывание и обновление	Сервис предъявляет актуальную цепочку	Старый сертификат остался на части узлов
Отзыв	CA публикует сведения об отзыве	Клиент проверяет их по своей политике

> CRL содержит список отозванных сертификатов. OCSP сообщает статус сертификата.

Параметры при установлении TLS

Поле / механизм	Что нужно выбрать
Версия TLS	Общий поддерживаемый вариант протокола
Key share	Параметры для согласования секрета
SNI: Server Name Indication	Имя сервиса на общем адресе
ALPN: Application-Layer Protocol Negotiation	Прикладной протокол, например h2 или http/1.1

> SNI помогает выбрать сервис. Проверка имени в сертификате выполняется отдельно.

Полный handshake TLS 1.3

Кто отправляет	Сообщения	Смысл
Клиент → сервер	ClientHello	Предложения параметров и key share
Сервер → клиент	ServerHello	Выбранные параметры и key share
Сервер → клиент	EncryptedExtensions, Certificate, CertificateVerify, Finished	Согласованный протокол, сертификат и доказательства
Клиент → сервер	Finished	Подтверждение handshake
Обе стороны	Application Data	Защищённые прикладные данные

> После ServerHello последующие handshake-сообщения защищены выведенными ключами.

Долгосрочный ключ и ключи сеанса

Ключ	Назначение
Приватный ключ сертификата	Подпись handshake, доказательство владения
Временные ключи (EC)DHE	Согласование секрета конкретного соединения
Симметричные ключи трафика	Защита данных в каждом направлении

Forward Secrecy: поздняя утечка долгосрочного ключа сама по себе не раскрывает записанный ранее обмен.

> Это свойство способа согласования ключей, а не просто надпись «TLS».

Граница защиты TLS

Что TLS защищает	Что требует отдельного внимания
Содержимое между двумя TLS-узлами	Данные в логах, памяти, хранилищах
Целостность защищённого обмена	Правильность JSON и бизнес-логики
Идентичность проверяемой стороны	Авторизация пользователя в приложении
Прикладные данные внутри соединения	IP-адреса, объём и время передачи

> Если TLS заканчивается на прокси, следующий участок соединения рассматриваем отдельно.

Разбор: три отказа TLS-клиента

Наблюдение	Что проверить	Чего ещё не знаем
Сертификат просрочен	notAfter и часы клиента	На всех ли узлах такой сертификат
Имя не совпадает	URL, SNI, SAN	Почему выбран именно этот сертификат
Не удаётся построить доверенную цепочку	Intermediates и trust store	Сервер или конфигурация клиента требуют исправления

Какой артефакт попросите у коллеги для каждого случая?

> Сохраняем точный текст ошибки, имя, время и предъявленную цепочку.

Версии TLS и совместимость

Версия	Как рассматриваем на курсе
TLS 1.0 / 1.1	Устаревшие версии. Не используем в учебном защищённом обмене
TLS 1.2	Отдельная версия с несколькими способами согласования ключей
TLS 1.3	Основная модель handshake на этой лекции

> Версия, алгоритмы и проверка сертификата решают разные вопросы совместимости.

Версии HTTP: общий смысл, разная передача

Методы, статусы и смысл полей сохраняются. Меняются кодирование и организация обмена.

Свойство	HTTP/1.1	HTTP/2	HTTP/3
Сообщения	Стартовая строка и текстовые поля	Бинарные фреймы	Бинарные фреймы
Обмены	Ответы по порядку в соединении	Потоки в одном TCP	Потоки QUIC
Заголовки	Текст	HPACK	QPACK
HTTPS	TLS поверх TCP	TLS поверх TCP	QUIC с TLS 1.3 поверх UDP

> Текстовое тело JSON возможно во всех трёх версиях.

HTTP/2 разделяет обмены на потоки

Фреймы от сервера клиенту	Какому обмену принадлежат
HEADERS, stream 1	Заголовки ответа страницы
HEADERS, stream 3	Заголовки ответа изображения
DATA, stream 1	Часть ответа страницы
DATA, stream 3	Часть ответа изображения

Фреймы разных потоков можно чередовать. Сжатие заголовков HPACK отдельно от gzip тела.

> Упорядоченная доставка TCP всё ещё может задержать несколько потоков.

HTTP/3 и отдельная тема QUIC

Здесь фиксируем	Подробно на лекции 4
HTTP/3 использует QUIC поверх UDP	Надёжность, управление потоком и перегрузкой
QUIC предоставляет потоки	Потери и границы блокировки
QUIC использует TLS 1.3 handshake	Установка соединения, 1-RTT и 0-RTT
У соединения есть идентификаторы	Connection ID и смена сетевого пути
Нужна доступность нужного протокола	Обнаружение HTTP/3 и fallback

> QUIC использует TLS handshake, но защищает пакеты без TLS record layer.

Диагностика: этап, факт, следующая проверка

Наблюдение	Граница	Следующая проверка
MIME-вложение не извлекается	Структура составного сообщения	Boundary и метаданные части
Не доверяем сертификату	Проверка другой стороны	Имя, цепочка, время
Тело короче Content-Length	Полнота HTTP-сообщения	Полученные байты и завершение соединения
JSON-парсер отвергает данные	Разбор представления	Полнота, кодирование и фактическое тело

> Название ошибки помогает выбрать проверку, но не назначает виновника.

Проверка понимания

Клиент подключился по HTTPS, получил `200 OK`, но не смог прочитать ожидаемый JSON.

1. Что подтверждает успешная проверка TLS?
2. Что нужно проверить до запуска JSON-парсера?
3. Какие причины остаются, если всё тело получено?
4. Какие данные помогут коллеге воспроизвести проблему?

> Ответ должен отделять наблюдение от предположения.

Практика после лекции

Материал	Задача
Исходник письма	Разметить части и восстановить текст
HTTP-ответ	Определить границы и порядок обработки тела
Сертификат и сообщение клиента	Найти нарушенную проверку
Диагностический отчёт	Отделить факты от гипотез

В лонгриде есть примеры для самостоятельного запуска и справочник по сокращённым темам.